

VaiJunto

Roteiro de apresentação

TEC502 • Problema 1 • Falas, demonstração e apoio no código

Roteiro organizado pelos 11 critérios do barema, para apresentar a solução ao professor e aprofundar as decisões quando houver perguntas.

COMO USAR

Leia os blocos de fala para ensaiar. Use as explicações complementares quando o professor pedir detalhes e abra o arquivo indicado para mostrar a implementação. Os caminhos são relativos à pasta VAIJUNTO; as linhas correspondem ao código consultado na preparação e podem mudar após edições.

AJUSTE IMPORTANTE SOBRE TCP E UDP

O projeto usa TCP em todas as operações e não implementa telemetria por UDP. TCP entrega um fluxo ordenado de bytes, mas não garante sozinho que uma reserva seja executada exatamente uma vez. A chave da operação trata reenvios; a trava e a validação integral garantem a atomicidade no servidor.

ORDEM DA APRESENTAÇÃO

1. Arquitetura e modelo de dados
2. Comunicação TCP
3. Protocolo remoto
4. Encapsulamento e validação
5. Busca de itinerários
6. Concorrência
7. Atomicidade da reserva
8. Interação e demonstração
9. Confiabilidade e falhas
10. Testes e carga
11. Docker e máquinas distintas

BASE E LIMITES

Fonte: TEC502 - Problema 1 - VaiJunto - Barema.pdf, páginas 1 e 2, e implementação do projeto. Preparado em 17/09/2026. Os comandos são sugestões para a demonstração; não foram executados testes nem medidas de desempenho na preparação deste roteiro. Apresente resultados reais da execução que realizar.

1. Arquitetura e modelo de dados

FALA PRINCIPAL

“Eu desenvolvi o VaiJunto em Go, com um servidor central e dois tipos de cliente: motorista e passageiro. O motorista publica caronas e acompanha os passageiros por trecho. O passageiro busca itinerários, confirma reservas e pode cancelá-las antes do início da viagem. O servidor concentra o estado e as regras do sistema. Os clientes coletam informações e enviam pedidos; quem decide se ainda existe vaga e se o usuário possui permissão é o servidor.”

SE O PROFESSOR PEDIR A JUSTIFICATIVA

“Escolhi centralizar as decisões para que dois passageiros não mantenham visões independentes da mesma vaga. Todos consultam e alteram o mesmo estado, protegido contra acessos concorrentes. Os dados ficam em mapas na memória. Essa escolha simplifica a implementação, mas implica que reiniciar o servidor perde os cadastros, caronas e reservas.”

MOSTRE NO CÓDIGO

internal/servidor/gerenciador.go • linha 49

Grafotinerarios reúne os mapas e a trava do estado central.

```
usuarios map[string]conta
sessoes map[string]sessao
caronas map[string]*oferta
trechos map[string]protocolo.Trecho
reservas map[string]protocolo.Reserva
```

EXPLIQUE OS NOMES DO MODELO

“A carona é a oferta de um motorista. A rota é a sequência de cidades. Cada par consecutivo de cidades forma um trecho, com preço, horário e vagas próprios. O itinerário é a sequência de trechos escolhida pelo passageiro, podendo combinar motoristas diferentes. O número de assento é atribuído automaticamente pelo servidor e controla a ocupação dentro de cada trecho.”

MOSTRE NO CÓDIGO

internal/protocolo/modelos.go • linha 85

Mostre PublicacaoCarona e depois Trecho, Carona, Itinerario e Reserva. Usuários são localizados pelo e-mail; reservas, pelo ID.

2. Comunicação: por que TCP

FALA PRINCIPAL

“Utilizei sockets TCP porque cadastro, publicação e confirmação de reserva precisam de uma comunicação ordenada e com detecção de falhas. O TCP cuida de retransmissões e da entrega ordenada dos bytes enquanto a conexão funciona. Cada operação do cliente abre uma conexão, envia o pedido, recebe a resposta e fecha a conexão. A sessão do usuário é identificada por um token, então fechar o socket não significa sair da conta.”

MOSTRE NO CÓDIGO

internal/cliente/tcp.go • linha 33

DialTimeout estabelece a conexão. O defer fecha o socket quando a função termina.

```
conn, err := net.DialTimeout(
    "tcp", endereco, 5*time.Second,
)
// Após verificar err:
defer conn.Close()
```

SE PERGUNTAR SOBRE UDP

“UDP poderia ser considerado em um sistema de localização frequente, no qual uma atualização nova substitui a anterior e alguma perda é aceitável. Esse projeto não implementa telemetria. Nas operações que implementei, escolhi TCP para evitar desenvolver manualmente mecanismos de transporte confiável.”

SE PERGUNTAR SE TCP GARANTE A RESERVA

“Ele não garante sozinho que a operação foi executada exatamente uma vez. O servidor pode confirmar a reserva e a resposta se perder. Por isso também implementei identificação de operações repetidas.”

CHEGADA E SAÍDA DOS CLIENTES

O servidor aceita sockets com Accept e atende cada conexão. O cliente oficial fecha o socket após cada resposta, mas a sessão continua válida. A saída pelo menu solicita DESCONECTAR; a queda abrupta do cliente deixa a sessão existir até expirar. O servidor também consegue ler várias mensagens sequenciais em um socket enquanto ele estiver aberto.

MOSTRE NO CÓDIGO

internal/servidor/tcp.go • linha 74

Servir aceita conexões; AtenderConexao, na linha 23, lê mensagens e fecha o socket ao sair.

3. Protocolo: mensagens e autenticação

FALA PRINCIPAL

“Sobre o TCP, defini um protocolo próprio com mensagens JSON delimitadas por quebra de linha, chamado NDJSON. A requisição possui ação, sessão e dados. A resposta possui status, mensagem e, quando necessário, dados. As constantes do pacote protocolo fazem cliente e servidor usarem os mesmos nomes de operação.”

MOSTRE NO CÓDIGO

internal/protocolo/modelos.go • linha 35

Requisicao e Resposta definem o envelope de comunicação.

```
type Requisicao struct {
    Acao      string      `json:"acao"`
    SessaoID  string      `json:"sessao,omitempty"`
    Dados     json.RawMessage `json:"dados"`
}
```

EXEMPLO DE CONFIRMAÇÃO

IDs ilustrativos. Na rede, este objeto ocupa uma linha e termina com LF; abaixo ele foi dividido apenas para facilitar a leitura.

```
{
  "acao": "CONFIRMAR_RESERVA",
  "sessao": "SESSAO_BIA",
  "dados": {
    "chave": "reserva-1",
    "trechos_ids": ["T0000002", "T0000004"]
  }
}
```

“Essa mensagem solicita uma única reserva envolvendo os dois trechos. Eu envio todos os IDs juntos porque a confirmação precisa ser integral.”

```
{"status": "ERRO",
 "mensagem": "trechos sem conexão espacial ou temporal"}
```

OPERAÇÕES E SESSÃO

A API inclui cadastro, autenticação, saída, publicação, busca, confirmação, consulta de caronas e reservas, cancelamento de carona/reserva/trecho, consulta de notificações e confirmação de leitura.

“O login devolve uma sessão aleatória com validade de 15 minutos. Nas próximas operações, o servidor verifica tanto a validade da sessão quanto o perfil exigido.”

MOSTRE NO CÓDIGO

internal/protocolo/PROTOCOLO.md

Especificação completa, campos e exemplos. Autenticar fica em internal/servidor/gerenciador.go:121; autorizar, na linha 153.

4. Encapsulamento e validação

FALA PRINCIPAL

“Escolhi JSON para enviar uma representação textual dos dados, em vez da memória bruta das estruturas Go. Assim, cliente e servidor concordam pelos nomes e tipos dos campos, independentemente do layout de memória da máquina. As tags json definem os nomes que aparecem na rede. A codificação é UTF-8 e as datas incluem fuso horário no formato RFC3339.”

POR QUE EXISTE UMA QUEBRA DE LINHA?

“TCP transporta um fluxo de bytes. Ele não preserva uma fronteira entre cada pedido: uma mensagem pode chegar em várias leituras. Por isso uso a quebra de linha para reconhecer quando a mensagem está completa.”

MOSTRE NO CÓDIGO

internal/servidor/tcp.go • linha 23

AtenderConexao aguarda uma linha completa antes de chamar o roteador.

```
linha, err := leitor.ReadSlice('\n')
```

O QUE ACONTECE NA CHEGADA?

“O servidor verifica se recebeu um objeto JSON válido em UTF-8, rejeita campos desconhecidos e chaves duplicadas. Depois converte os dados para a estrutura correspondente à ação. As validações de negócio conferem campos, sessão, perfil, horários e disponibilidade.”

MOSTRE NO CÓDIGO

internal/servidor/roteador.go • linha 58

decodificar valida formato; processar escolhe a operação; ProcessarMensagem padroniza SUCESSO ou ERRO.

```
d := json.NewDecoder(bytes.NewReader(dados))
d.DisallowUnknownFields()
return d.Decode(alvo)
```

TAMANHO E ERROS

“Também limito o tamanho: a requisição JSON deve ter menos de 1 MiB, reservando espaço para o delimitador, e o cliente limita a resposta recebida a 16 MiB. Isso evita aceitar mensagens arbitrariamente grandes. Uma recusa de negócio retorna ERRO; uma conexão interrompida pode impedir que qualquer resposta seja recebida.”

1 MiB corresponde a 1.048.576 bytes. O sucesso de uma escrita no socket não equivale à confirmação de uma reserva pela aplicação.

5. Busca de itinerários

FALA PRINCIPAL

“Modelei a rede como um multigrafo direcionado: cidades são vértices e trechos são conexões com sentido. É um multigrafo porque podem existir várias ofertas entre as mesmas cidades, com motoristas, preços e vagas diferentes. Para buscar itinerários, uso busca em profundidade, ou DFS. O algoritmo segue possibilidades de saída de cada cidade e volta para experimentar outros caminhos.”

MOSTRE NO CÓDIGO

internal/servidor/caronas.go • linha 179

Buscar filtra trechos e prepara a exploração. A função recursiva explorar começa na linha 213.

EXEMPLO PARA EXPLICAR A CONEXÃO

“Para viajar de Salvador a Serrinha, o algoritmo pode combinar Salvador-Feira de um motorista com Feira-Serrinha de outro. Para conectar, o destino anterior precisa ser a origem seguinte e o próximo motorista não pode sair antes da chegada e da parada exigida.”

MOSTRE NO CÓDIGO

internal/servidor/caronas.go • linha 172

conectam verifica compatibilidade de cidade e horário.

CICLOS E LIMITES

“Evito repetir cidades no mesmo caminho para impedir ciclos. Também estabeleci limites de exploração: até 50 mil passos, 100 resultados e 12 trechos por padrão, configuráveis até 20. Quando a exploração é limitada, a resposta informa isso; não afirmo que encontrei todas as alternativas.”

O mapa visitadas pertence ao caminho atual. Ao voltar de uma chamada recursiva, a cidade é removida desse mapa para poder aparecer em outro caminho. Esse retorno é chamado de backtracking.

ORDENAÇÃO, PREÇO E DURAÇÃO

“Os resultados encontrados são ordenados por partida, depois preço, duração, quantidade de trechos e IDs para desempate. Somo os preços em centavos inteiros. A duração vai da primeira partida até a chegada final, incluindo a espera nas conexões.”

MOSTRE NO CÓDIGO

internal/servidor/caronas.go • linha 253

sort.Slice define a ordem de apresentação dos itinerários encontrados.

6. Concorrência

FALA PRINCIPAL

“O servidor atende cada conexão em uma goroutine. Isso permite manter vários clientes em atendimento ao mesmo tempo. Um canal funciona como semáforo, limitando a 256 atendimentos ativos para controlar o consumo de recursos.”

MOSTRE NO CÓDIGO

internal/servidor/tcp.go • linha 74

Servir controla conexões e inicia atendimentos. Ao atingir o limite, fecha a nova conexão.

```
vagas := make(chan struct{}, 256)

// Trecho simplificado do atendimento:
go func() {
    // A implementação também libera recursos ao sair.
    AtenderConexao(conn, g, timeout)
}()
```

COMO PROTEJO OS DADOS?

“Atender conexões simultaneamente exige proteger os mapas compartilhados. Para isso, usei um sync.RWMutex: consultas podem compartilhar a trava de leitura, enquanto alterações usam trava exclusiva. A trava só protege acessos que seguem esse mesmo mecanismo.”

MOSTRE NO CÓDIGO

internal/servidor/gerenciador.go • linha 49

O campo mu protege o estado central. A trava da lista de sockets em Servir é separada.

SE PERGUNTAR SOBRE DESEMPENHO

“A trava global simplifica a consistência entre vagas, reservas e notificações, mas serializa alterações mesmo em rotas independentes. É uma escolha de simplicidade e corretude com um custo de escalabilidade. Para reduzir o tempo de bloqueio na busca, copio os trechos disponíveis sob trava e executo o algoritmo depois de liberá-la.”

MOSTRE NO CÓDIGO

internal/servidor/caronas.go • linha 192

A cópia é construída sob RLock e a exploração ocorre depois de RUnlock.

Uma busca não segura vaga e pode ficar desatualizada. Confirmar verifica a disponibilidade novamente. A aquisição da trava não garante atendimento por ordem de chegada. A implementação usa goroutines por conexão com limite; não é um pool fixo de trabalhadores.

7. Atomicidade da reserva

FALA PRINCIPAL

“A reserva de um itinerário é tudo ou nada. Se o passageiro precisa de dois trechos, ele não pode ficar com o primeiro reservado e o segundo indisponível. Minha confirmação possui duas fases dentro da mesma trava exclusiva: primeiro valido todos os trechos e escolho assentos livres; somente depois desconto as vagas e registro a reserva.”

MOSTRE NO CÓDIGO

internal/servidor/reservas.go • linha 27

Abra Confirmar e mostre a trava e os dois comentários de fase.

```
g.mu.Lock()
defer g.mu.Unlock()

/* Fase 1: somente verificacao */
// Valida todos os trechos e escolhe assentos.

/* Fase 2: confirmar todos os trechos */
// Desconta vagas e registra reserva e chave.
```

SE O ÚLTIMO TRECHO ESTIVER LOTADO

“A função retorna erro durante a primeira fase. Como ainda não alterei vagas, os trechos anteriores permanecem intactos. Não preciso desfazer descontos parciais nessa situação.”

Exemplo: A-B possui uma vaga e B-C está lotado. A tentativa de reservar A-B-C é recusada e A-B continua com uma vaga. Enquanto essa verificação ocorre, outra escrita não pode mudar o estado protegido pela mesma trava.

SE PERGUNTAR SOBRE DEADLOCK

“Não adquiero uma trava para cada trecho. Uso uma única trava para a transação inteira. Portanto, não existe nesse desenho o caso de um passageiro segurar A e esperar B enquanto outro segura B e espera A. O defer libera a trava nos retornos da função, inclusive os de erro, antes do envio da resposta pela rede.”

LIMITE DA GARANTIA

A atomicidade apresentada é a da operação sobre o estado em memória sob concorrência. Não é uma transação persistente capaz de sobreviver à queda do processo ou da máquina.

MOSTRE NO CÓDIGO

testes/concorrenca_test.go • linha 56

TestAtomicidadeEAssentoPorTrecho exercita essa propriedade.

8. Interação e demonstração

FALA PRINCIPAL

“Os clientes são menus de terminal. O cliente valida campos para orientar o usuário, mas o servidor repete as validações porque não pode confiar em uma entrada remota. No motorista, vou publicar e consultar caronas com passageiros por trecho. No passageiro, vou buscar uma conexão entre dois motoristas, confirmar, consultar e cancelar a reserva.”

MOSTRE NO CÓDIGO

internal/cliente/menu.go • linha 31

ExecutarMenu controla os menus. A publicação está em internal/cliente/motorista.go:17.

DADOS DA DEMONSTRAÇÃO

Use a mesma data futura para as duas ofertas e cadastre dois motoristas com e-mails diferentes.

Campo	Ana	Bruno
Rota	Salvador → Feira de Santana	Feira de Santana → Serrinha
Saída	08:00	10:30
Viagem	120 minutos	60 minutos
Vagas	2	2
Preço	R\$ 50,00	R\$ 40,00

FALA DURANTE A BUSCA

“A primeira viagem chega às 10 horas e a segunda sai às 10h30. Então a conexão é possível. O total é R\$ 90 e a duração é de três horas e meia, incluindo os 30 minutos de espera.”

1. Reserve Salvador → Serrinha.
2. Consulte ambos os motoristas: deve restar uma vaga em cada trecho.
3. Cancele pelo passageiro: devem voltar duas vagas em cada trecho.
4. Reserve novamente.
5. Cancele o trecho do segundo motorista.
6. Consulte a reserva e os avisos do passageiro.

“Cancelar o trecho desfaz a reserva inteira que dependia dele e devolve vagas nos dois trechos. A oferta independente do primeiro motorista continua ativa. O passageiro consulta os avisos; o servidor não envia notificações espontaneamente.”

MOSTRE NO CÓDIGO

internal/servidor/reservas.go • linha 124

cancelarReserva devolve vagas. O cliente consulta e confirma leitura em internal/cliente/notificacoes.go:18.

9. Confiabilidade e falhas

FALA PRINCIPAL

“Configurei cinco segundos para estabelecer conexão no cliente e um prazo de 30 segundos para a troca de dados. No servidor, há prazos de leitura e escrita. Isso impede que sockets aguardem indefinidamente uma mensagem.”

MOSTRE NO CÓDIGO

internal/cliente/tcp.go • linha 33

DialTimeout e SetDeadline. No servidor, AtenderConexao define prazos separados de leitura e escrita.

Os prazos são de operações de rede; eles não interrompem automaticamente um algoritmo de busca que esteja executando no servidor.

SE O CLIENTE FECHAR NO MEIO DA RESERVA

“Se a linha JSON estiver incompleta, o servidor não executa o pedido. Se a operação já foi concluída e apenas a resposta falhou, a reserva permanece. Não mantenho vaga temporariamente bloqueada enquanto espero confirmação pela rede. A trava existe apenas durante o processamento no servidor. Uma reserva concluída é uma ocupação válida e não é cancelada automaticamente pela queda do cliente.”

RESPOSTA PERDIDA E REENVIO

“Publicação e confirmação possuem uma chave de idempotência. Se o cliente reenviar a mesma chave com os mesmos dados, o servidor devolve o recurso anterior. Se usar a mesma chave com dados diferentes, recebe erro. Portanto, a proteção contra uma reserva duplicada por reenvio está na aplicação, além do TCP.”

MOSTRE NO CÓDIGO

internal/cliente/sessao.go • linha 31

A requisição é montada antes do laço de tentativas. Em internal/servidor/reservas.go:41, a chave é associada ao usuário e à operação.

LIMITAÇÕES QUE VOCÊ DEVE SABER EXPLICAR

“Essas garantias valem enquanto o estado do servidor permanece na memória. Não implementei persistência nem replicação. Também não há TLS: o hash protege o armazenamento da senha, mas não criptografa a conexão TCP.”

Cadastro não usa a chave de idempotência de publicação/reserva. Se sua resposta se perder, o usuário pode tentar autenticar com as credenciais cadastradas. Não apresente todas as operações como se tivessem exatamente a mesma política de reenvio.

10. Testes automáticos e carga

FALA PRINCIPAL

“Separei testes de regras, protocolo, integração por TCP, concorrência e falhas. Também testo limites de horário: um segundo antes da partida, no instante da partida e depois. Para concorrência, verifico a disputa pelo último assento e se uma falha em um trecho preserva os outros. Para reenvio, verifico se recuperar uma resposta perdida evita duplicação.”

MOSTRE NO CÓDIGO

testes/concorrencia_test.go • linha 18

Último assento; atomicidade na linha 56. Testes de resposta perdida: testes/falhas_test.go:20. Fluxo de menu via TCP: internal/cliente/menu_test.go:21.

COMANDOS PARA MOSTRAR

```
go test -count=1 -v ./...
```

```
# Com servidor ativo:
```

```
go run ./cmd/carga -servidor 127.0.0.1:8080 -clientes 100 -vagas 5
```

FALA DURANTE A CARGA

“Esse teste cria 100 passageiros disputando cinco vagas em um itinerário com dois motoristas. O resultado esperado é cinco confirmações, 95 recusas, nenhum erro de transporte e integridade verdadeira. Além das contagens, verifico vagas restantes e assentos únicos. Também meço latência média, P95 e requisições por segundo. O P95 é o tempo dentro do qual pelo menos 95% das tentativas medidas terminaram.”

MOSTRE NO CÓDIGO

internal/carga/carga.go • linha 76

Executar prepara o cenário e a concorrência. P95 na linha 209; integridade final na linha 234.

COMO INTERPRETAR OS RESULTADOS

“A medição cobre a fase concorrente de confirmação, sem incluir o cadastro dos usuários. As recusas esperadas também entram nas métricas. Vou apresentar os números obtidos nesta execução, porque variam conforme máquina e rede.”

Uma recusa por falta de vagas é esperada quando a capacidade acaba; erro de transporte é uma falha de comunicação. O detector -race procura acessos concorrentes indevidos à memória nos caminhos exercitados. Os testes de integridade verificam também regras de negócio que o detector, sozinho, não demonstra.

11. Docker e máquinas distintas

FALA PRINCIPAL

“Preparei um Dockerfile com etapas de testes, compilação e execução. A construção executa testes com detector de condições de corrida e go vet, compila os programas e monta uma imagem final com os executáveis. Isso ajuda a repetir o ambiente no laboratório sem instalar Go em cada máquina.”

MOSTRE NO CÓDIGO

Dockerfile • linha 1

Etapas testes, compilacao e runtime. O runtime usa scratch e executa como usuário não root.

```
RUN CGO_ENABLED=1 go test -race -v ./...  
RUN go vet ./...
```

MOSTRE NO CÓDIGO

compose.yaml • linha 1

O serviço servidor publica a porta; os clientes recebem VAIJUNTO_SERVIDOR.

```
ports:  
  - "${VAIJUNTO_PORTA:-8080}:8080"
```

CONECTIVIDADE ENTRE MÁQUINAS

“Na mesma rede do Compose, os clientes encontram o servidor pelo nome servidor:8080. Em máquinas distintas, uso o IP da máquina que hospeda o servidor e a porta publicada. Localhost dentro do cliente aponta para o próprio ambiente do cliente. Ele não identifica o outro computador. Por isso configuro o endereço remoto por variável de ambiente e verifico a liberação da porta TCP no firewall.”

NA MÁQUINA DO SERVIDOR

```
docker compose build  
docker compose up -d servidor
```

NA MÁQUINA DO CLIENTE • POWERSHELL

Com a imagem já construída nessa máquina, substitua o IP de exemplo pelo endereço real do servidor. Não suba outro servidor para essa demonstração.

```
$env:VAIJUNTO_SERVIDOR = "192.168.1.10:8080"  
docker compose run --rm passageiro
```

```
Test-NetConnection 192.168.1.10 -Port 8080
```

“Os contêineres padronizam o ambiente e permitem executar várias instâncias dos clientes. A comunicação entre computadores continua dependendo da rede, do IP correto, da porta publicada e do firewall.”

Encerramento e consulta rápida

FALA DE ENCERRAMENTO

“O ponto principal da solução é manter uma reserva consistente mesmo quando envolve diferentes motoristas e vários passageiros disputam vagas. Para isso, combinei transporte TCP, mensagens bem definidas, confirmação atômica sob trava e chaves para reconhecer reenvios. Agora posso demonstrar esses comportamentos no menu e nos testes.”

SE O PROFESSOR PERGUNTAR...

“TCP impede reserva duplicada?”

A chave da operação identifica reenvios. TCP não garante sozinho execução única da operação.

“Buscar já segura a vaga?”

Não. A busca é uma fotografia. Confirmar revalida tudo sob trava exclusiva.

“O último trecho lotou: o primeiro fica reservado?”

Não. A fase de validação termina com erro antes de qualquer desconto.

“Por que não ocorre deadlock entre trechos?”

A confirmação usa uma única trava global, em vez de adquirir várias travas de trechos.

“Fechar o cliente cancela a reserva?”

Não. Socket, sessão e reserva têm ciclos de vida diferentes. A reserva concluída permanece.

“Notificação chega automaticamente?”

O cliente consulta o servidor e marca o aviso como lido depois de mostrá-lo.

“A busca sempre encontra o melhor caminho?”

Há limites de exploração. A ordenação vale para o conjunto encontrado; limitada avisa quando a exploração foi restringida.

“E se o servidor reiniciar?”

O estado está em RAM e é perdido, inclusive as chaves de repetição.

“A senha está criptografada na rede?”

Não há TLS. Salt e hash protegem o armazenamento; não a transmissão.